

Estructuras de Datos — Quiz 03

Dynamic Arrays and Amortized Analysis · analysis.pdf · Semestre 2026-2

Nombre: Kevin Javier Gonzalez

Documento: 1093432420

Grupo: 3

Notación y modelo de costo

P1 — Arreglo dinámico desde cero

a) **Implementación.** En el archivo de `DynamicArray.java` esta el Código.

b) **Invariantes.**

I1. $0 \leq \text{size} \leq \text{capacity}$

I2. $\text{capacity} \geq 1$

I3. valores no nulos en `data` estan desde $[0, \text{size} - 1]$, `data[size] == null`

c) **Complejidad worst-case.**

Operación	Worst-case	Justificación
<code>get</code>	$\Theta(1)$	es simplemente una búsqueda, el array por defecto ya lo hace en $O(1)$
<code>set</code>	$\Theta(1)$	igualmente al <code>set</code> , es una búsqueda de costo 1 + una asignación de costo 1 en todo los casos
<code>append</code>	$O(n)$	en el peor caso tengo que construir un nuevo array y dato por dato del anterior asignarlo en la misma pos del nuevo array, claro n es <code>size</code> dado que mas alla de <code>size</code> no toca asignar nada
<code>removeLast</code>	$\Theta(1)$	aca igualmente es siempre el mismo valor, hago una búsqueda de costo 1, y hago una asignación de costo 1, y luego simplemente hago <code>size--</code> que tambien es de costo 1

d) **Worst-case vs. amortizado.**

cuando hago `append` si $\text{size} < \text{capacity}$ entonces solo tengo que hacer una operacion de asignacion de coste 1, entonces cada que se duplica la capacity tengo $\frac{\text{capacity}}{2}$ veces para hacer operaciones de costo 1

P2 — Snake sobre `DynamicArray` con desplazamiento

a) **Costo de `removeTail`.** cuando se llama a `remove tail` en esa implementacion, toca eliminar la cola y luego, reasignar los valores que quedan una posicion hacia atras, el costo seria $O(n-1) \rightarrow O(n)$, con n siendo el `size`.

b) **Costo total de m movimientos.** siendo de longitud k , hacer m movimientos seria que en cada movimiento tengo el costo $O(k)$ de mover la serpiente, m veces esto, osea el costo final seria $O(k \cdot m)$

c) **Validez de la afirmación.** esto es falso, `move()` no es amortizado porque en cada operacion de `move()` tengo que hacer n (`size`) actualizaciones de posicion de la serpiente lo que hace que en promedio sea $\frac{O(n \cdot n)}{n} \rightarrow O(n)$ veces el costo amortizado, a diferencia del `append`, que en n operaciones el coste es $\frac{O(2n) + O(n)}{n} \rightarrow O(3) \rightarrow O(1)$

d) **Bottleneck.** el bottleneck seria que como en cada movimiento que no coma tengo que hacer `eliminarCola()` entonces tendria n para ajustar el array dado que la nueva cola debe estar en la posicion 0, pero a su vez como esta en movimiento tengo que hacer n operaciones de calcular su nueva posicion, esto haria que para m operaciones sin comer tendria $(n + n) \cdot m$ operaciones

P3 — Rediseño: representación interna

a) Variables de estado.

- data:
- start:
- size:
- capacity:

b) Mapeo lógico $\rightarrow$ físico.

$$\text{phys}(i) =$$

c) Operaciones.

- addHead:
- removeTail:
- get:

d) resize.

e) Invariantes y corrección.

S1.

S2.

S3.

S4.

f) Traza.

Mov.	Arreglo físico	start	size	capacity	¿resize?	Orden lógico
inicial						
N1 (no come)						
N2 (come)						
N3 (no come)						
N4 (no come)						
N5 (come)						
N6 (no come)						

P4 — Análisis amortizado

a) Aggregate Method.

b) Suma geométrica.

c) Costo total y amortizado de addHead.

d) Papel de m y g .

e) Método del potencial.

$$\Phi(\text{size}, C) =$$

- addHead sin resize:
- resize:
- removeTail:

P5 — The Adversarial Snake

a) El adversario no cambia el número de copias.

b–e) Comparación de políticas.

Política	# Resizes	Total copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
$C + 1$				
$2C$				
$\lceil 1.5C \rceil$				

f) Trade-off.

Familia $C_{\text{new}} = \lceil \alpha C \rceil$.

- Valores de α :
- Progreso ($C_{\text{new}} > C$):
- Cota superior de copias:
- Cota inferior de copias:
- Costo amortizado:
- Necesidad de $\alpha > 1$ constante: